```
======================================================================

Task 1: Single Digit Binary
Build a grammar that accepts exactly one binary digit, either 0 or 1.

<bit> ::= "0" | "1"
======================================================================


Task 2: Binary Strings (Non-empty)
Build a grammar that accepts any non-empty binary string.

<binary> ::= <bit> | <bit> <binary>
<bit> ::= "0" | "1"

======================================================================


Task 3: Even Number of a Characters
Build a grammar over {a,b} where the count of a is even.

<even> ::= ε | "b" <even> | "a" <odd>
<odd> ::= "b" <odd> | "a" <even>

======================================================================


Task 4: Identifier Names
Build a grammar for identifiers that start with a letter, followed by letters or digits.

<identifier> ::= <letter> <rest>
<rest>       ::= ε | <letter> <rest> | <digit>  <rest>
<letter>     ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n"
| "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z"
<digit>      ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

======================================================================

Task 5: Unsigned Integer
Build a grammar for unsigned base-10 integers with no sign.

<unsigned> ::= <digit> | <digit> <unsigned>
<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

======================================================================


Task 6: Simple Arithmetic (Plus Only)
Build a grammar that accepts numbers joined by +, for example 1+2+3, with no spaces.

<expr> ::= <unsigned> | <unsigned> "+" <expr>
<unsigned> ::= <digit> | <digit> <unsigned>
<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

======================================================================


Task 7: Matched Parentheses
Build a grammar that accepts correctly balanced parentheses using only ( and ).

<parens> ::= ε | <parens> <parens>  | "(" <parens> ")"

======================================================================


Task 8: Comma-Separated Integer List
Build a grammar for one or more integers separated by commas, with no trailing comma.

<list> ::= <unsigned> | <unsigned> "," <list>
```

```
<unsigned> ::= <digit> | <digit> <unsigned>
<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

==========

Task 9: Binary Multiples of 2
Build a grammar for binary integers that are multiples of 2 (so the last bit must be 0).

```
<multiple-of-2> ::= <prefix> "0"
<prefix> ::= ε | <bit> <prefix>
<bit> ::= "0" | "1"
```

==========

Task 10: Signed Decimal Number
Build a grammar for signed decimal numbers: optional + or -, digits, and optional fractional part.

```
<number> ::= <opt-sign> <unsigned> <opt-frac>
<opt-sign> ::= ε | "+" | "-"
<opt-frac> ::= ε | "." <unsigned>
<unsigned> ::= <digit> | <digit> <unsigned>
<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

==========

Task 11: Mini Expression Parser
Build a grammar for arithmetic expressions supporting unsigned integers, addition (+), multiplication (*), parentheses, correct operator precedence (examine the parse tree to check) and left associativity (2+3+4 should be parsed as (2+3)+4 and not 2+(3+4))

```
<expr> ::= <expr> "+" <term> | <term>
<term> ::= <term> "*" <factor> | <factor>
<factor> ::= <unsigned> | "(" <expr> ")"
<unsigned> ::= <digit> | <digit> <unsigned>
<digit> ::= "0" | "1" | "2" | "3" | "4"
          | "5" | "6" | "7" | "8" | "9"
```

==========

Task 12: Email address
Build a grammar that accepts valid email addresses.

```
<email> ::= <local-part> "@" <domain>

<local-part> ::= <local-char>
               | <local-char> <local-part>

<local-char> ::= <letter>
               | <digit>
               | "."
               | "_"

<domain> ::= <label> "." <domain>
           | <tld>

<label> ::= <alnum>
          | <alnum> <label>

<tld> ::= <letter> <letter>
        | <letter> <tld>

<alnum> ::= <letter> | <digit>

<letter> ::= <upper> | <lower>

<lower> ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n" |
```

```
"o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z"

<upper> ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" | "N" |
"O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z"

<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

==========

Task 13: Floating point numbers.
Build a grammar that accepts valid floating point numbers.

```
<number> ::= <opt-sign> <mantissa> <opt-exp>

<opt-sign> ::= ε | "+" | "-"

<mantissa> ::= <unsigned>
             | <unsigned> "." <unsigned>

<opt-exp> ::= ε
            | <exp>

<exp> ::= "E" <opt-sign> <unsigned>
        | "e" <opt-sign> <unsigned>

<unsigned> ::= <digit>
             | <digit> <unsigned>

<digit> ::= "0" | "1" | "2" | "3" | "4"
          | "5" | "6" | "7" | "8" | "9"
```

==========